STUDENT MANAGEMENT SYSTEM

Using Mongoose, Express.js and MongoDB

PROJECT / ASSIGNMENT REPORT

1. Aim

To develop a Student Management System using Node.js, Express.js, Mongoose, and MongoDB that allows student and faculty records to be created, viewed, updated, and deleted through REST API endpoints.

2. Objectives

- To understand server-side development using Node.js and Express.js.
- To connect an application to MongoDB Atlas using Mongoose.
- To design Student and Faculty data models.
- To implement REST API operations (Create, Read, Update, Delete).
- To test API requests and responses using Postman.
- To provide a foundation for a frontend with Student and Faculty sections.

3. Introduction

A Student Management System is an application used to store and manage student and faculty information. This project demonstrates a backend built with Node.js and Express.js, with MongoDB Atlas used to store records. Mongoose defines the database schemas and provides a convenient way to interact with MongoDB. Postman can be used to send requests and verify API responses.

4. Software and Hardware Requirements

Category	Requirement
Operating System	Windows, macOS, or Linux
Runtime	Node.js and npm
Code Editor	Visual Studio Code
Database	MongoDB Atlas
API Testing	Postman
Libraries	Express.js, Mongoose, dotenv, Nodemon (development)
Hardware	Computer with internet connection

5. Technologies Used

- Node.js: JavaScript runtime for the server.
- Express.js: Framework for creating routes and REST API endpoints.
- MongoDB Atlas: Cloud-hosted NoSQL database.
- Mongoose: Object Data Modeling library for schemas and database operations.
- dotenv: Loads configuration values from a local .env file.
- Postman: Used to test HTTP requests and responses.

6. System Architecture

Layer	Role
Frontend / Postman	Sends HTTP requests
Express.js REST API	Handles routes and validation
Mongoose	Defines models and performs database operations
MongoDB Atlas	Stores Student and Faculty records

7. Modules

7.1 Student Module

Manages student records. Typical fields include name, email, roll number, course, semester, and an optional faculty reference. The API may provide create, list, view-by-ID, update, and delete operations.

7.2 Faculty Module

Manages faculty information such as name, email, department, and designation. Student records may be associated with a faculty member through a faculty ID.

7.3 Database Module

MongoDB Atlas stores the application data in collections. Mongoose schemas help define field types, validation, uniqueness, and relationships between records.

7.4 API Testing Module

Postman can be used to send GET, POST, PUT, and DELETE requests and inspect status codes and response bodies.

8. Database Design

Collection	Fields (typical)	Purpose
Student	name, email, rollNo, course, semester, facultyId, timestamps	Stores student details and optional faculty association.
Faculty	name, email, department, designation, timestamps	Stores faculty details.

Note: Confirm the exact field names and validation rules in your current model files before submitting the report.

9. Implementation Overview

- Create Student and Faculty schemas in the models directory.
- Define REST API routes in the routes directory.
- Load configuration values such as the MongoDB URI and port from environment variables.
- Connect to MongoDB Atlas before accepting database-dependent requests.
- Start the Express server on the configured local port (the project has been set up to use port 4000).
- Use Postman to test endpoints and inspect returned JSON.

10. API Endpoints

The table below is a suggested endpoint checklist based on the project structure discussed. Verify each route in the current code before treating it as implemented.

Method	Endpoint	Purpose
GET	/api/students	List students, if implemented.
POST	/api/students	Create a student.
GET	/api/students/:id	Get a student by ID.
PUT	/api/students/:id	Update a student, if implemented at this path.
DELETE	/api/students/:id	Delete a student, if implemented at this path.
GET / POST	/api/faculty	List or create faculty records.
PUT / DELETE	/api/faculty/:id	Update or delete a faculty record.
GET / PUT / DELETE	/api/faculty/students[/:id]	Student operations under the faculty router, if implemented.

11. Testing

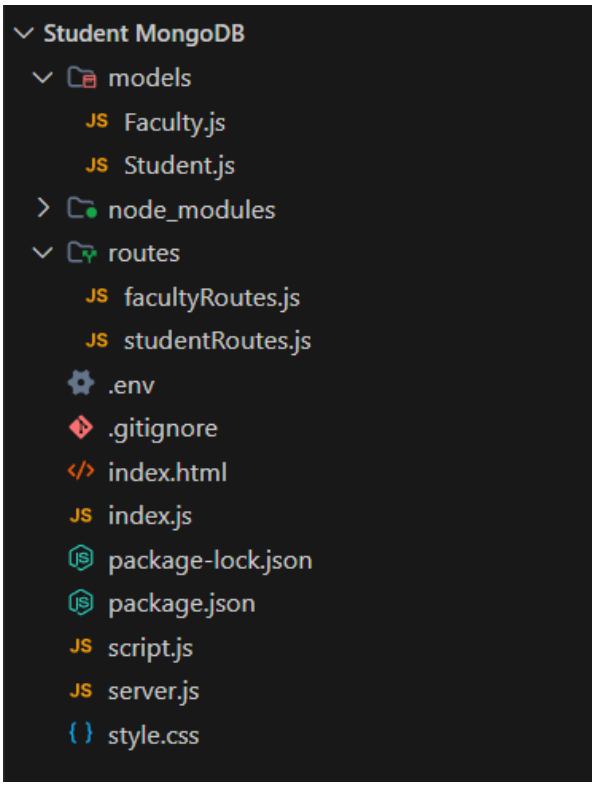
Test the application locally using Postman. Use valid JSON request bodies for POST and PUT requests. Save the actual response and status code for each test. Do not report a test as successful unless it has been run against the current application.

Test	Expected Result	Observed Result
Create student	201 Created or documented success code	Fill after testing
Retrieve student	200 OK with matching student data	Fill after testing
Create faculty	201 Created or documented success code	Fill after testing
Update / delete record	Success response and updated database state	Fill after testing
Duplicate unique field	Clear validation/conflict error	Fill after testing

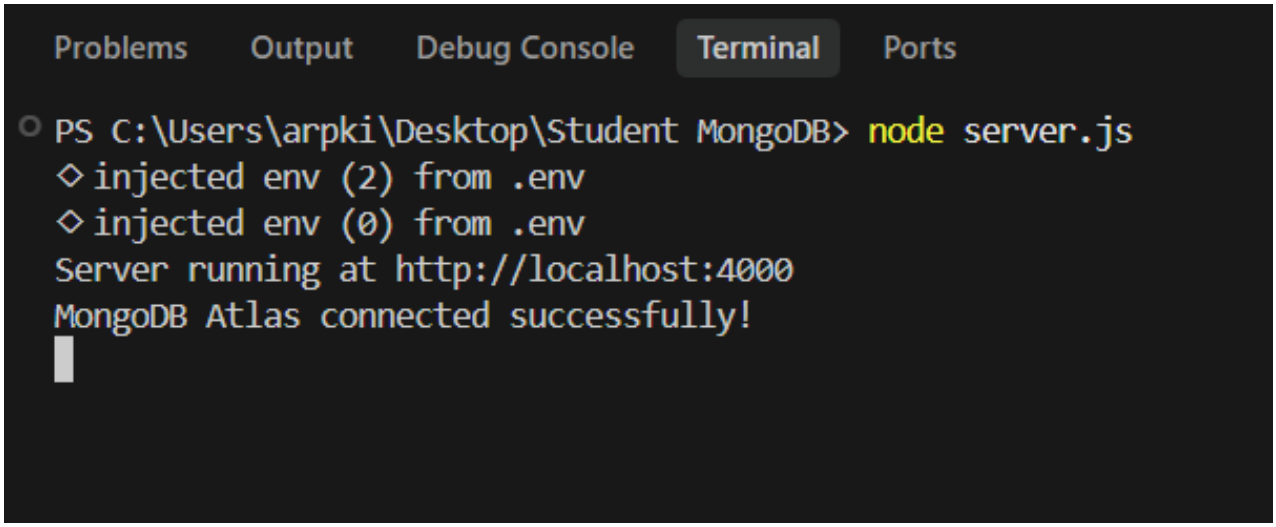
12. Screenshots and Observations

Insert your own screenshots in the spaces below. These are placeholders, not real screenshots from your computer or Postman session.

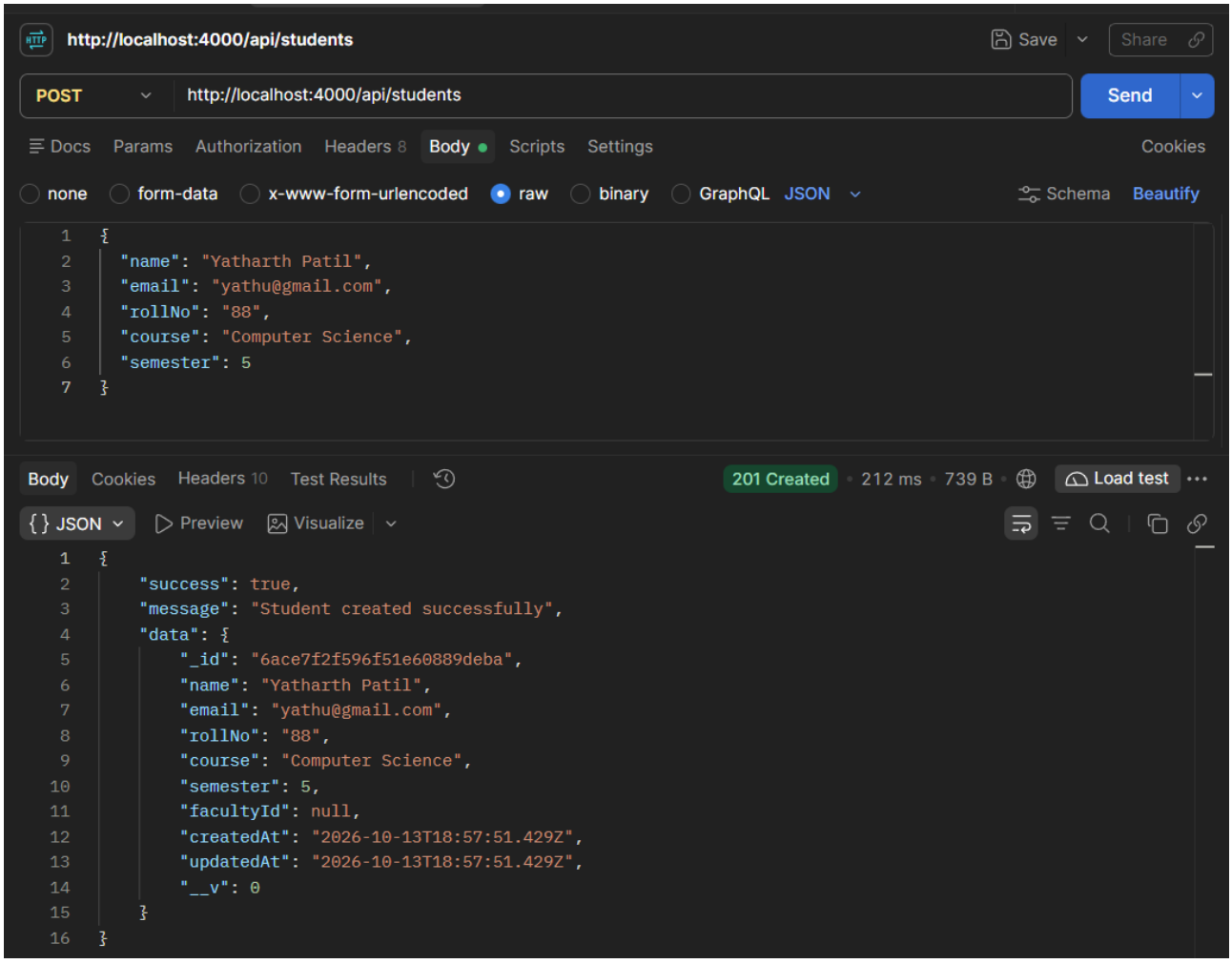
Screenshot 1: VS Code project structure



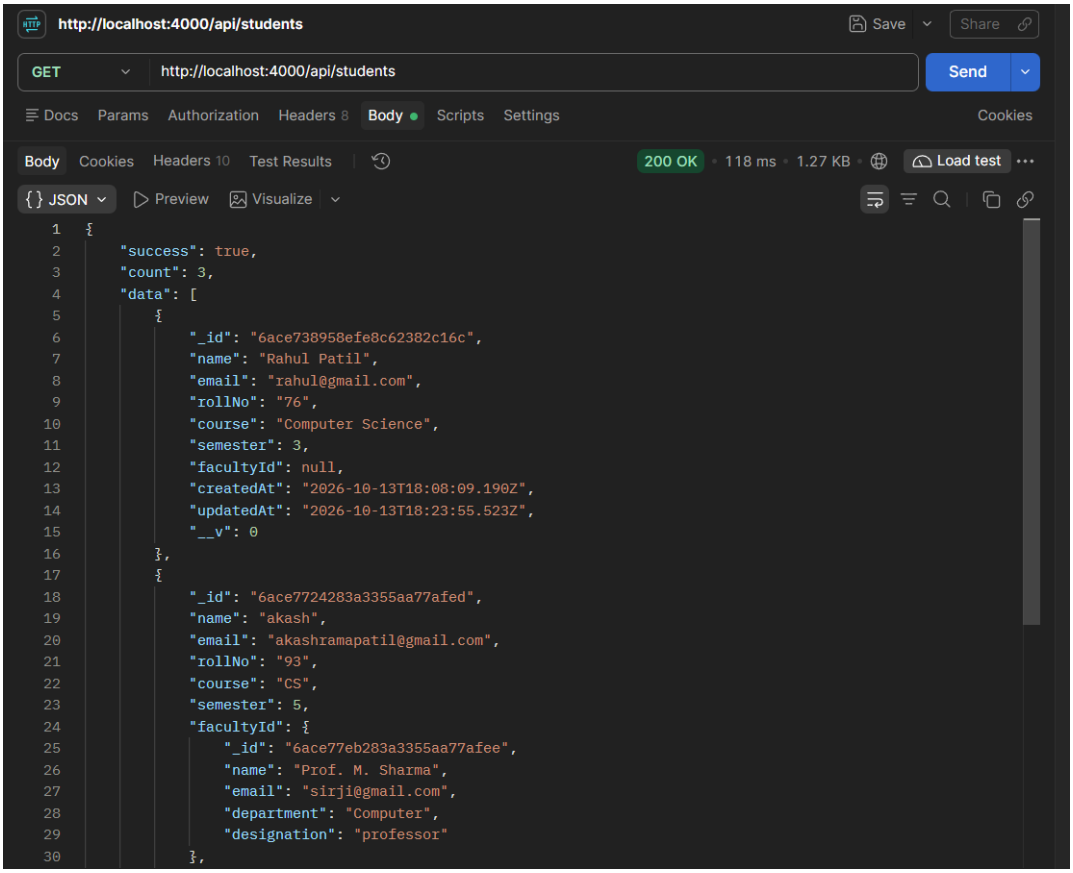
Screenshot 2: MongoDB connection



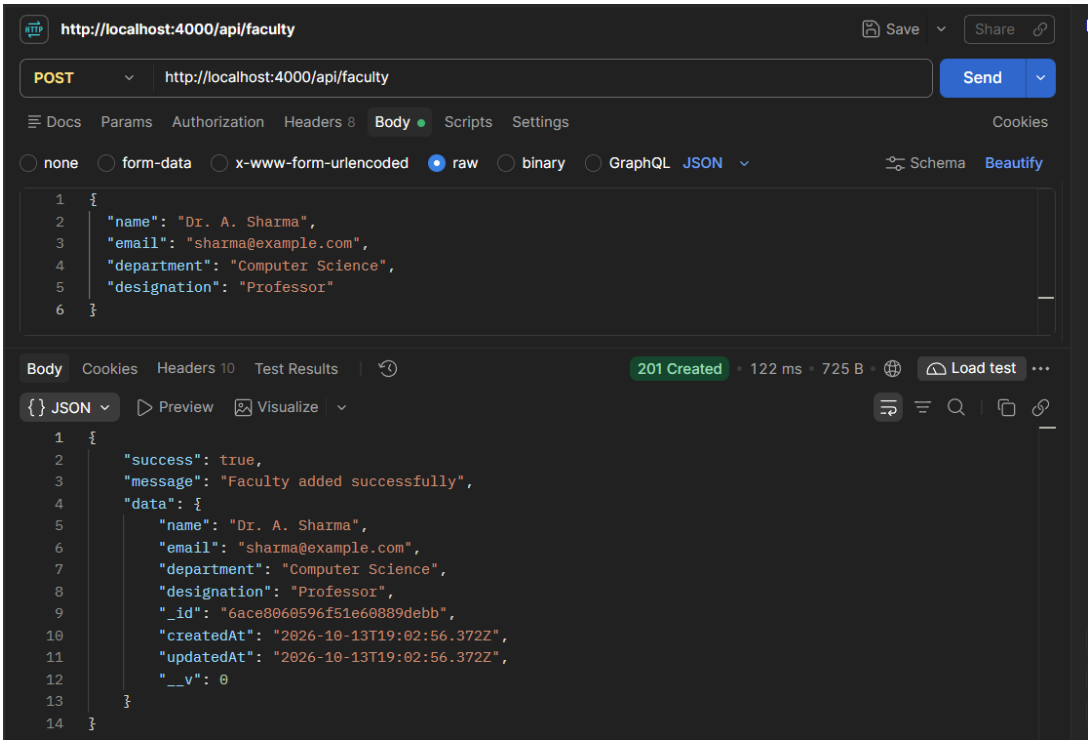
Screenshot 3: Create student in Postman



Screenshot 4: Retrieve student or list students

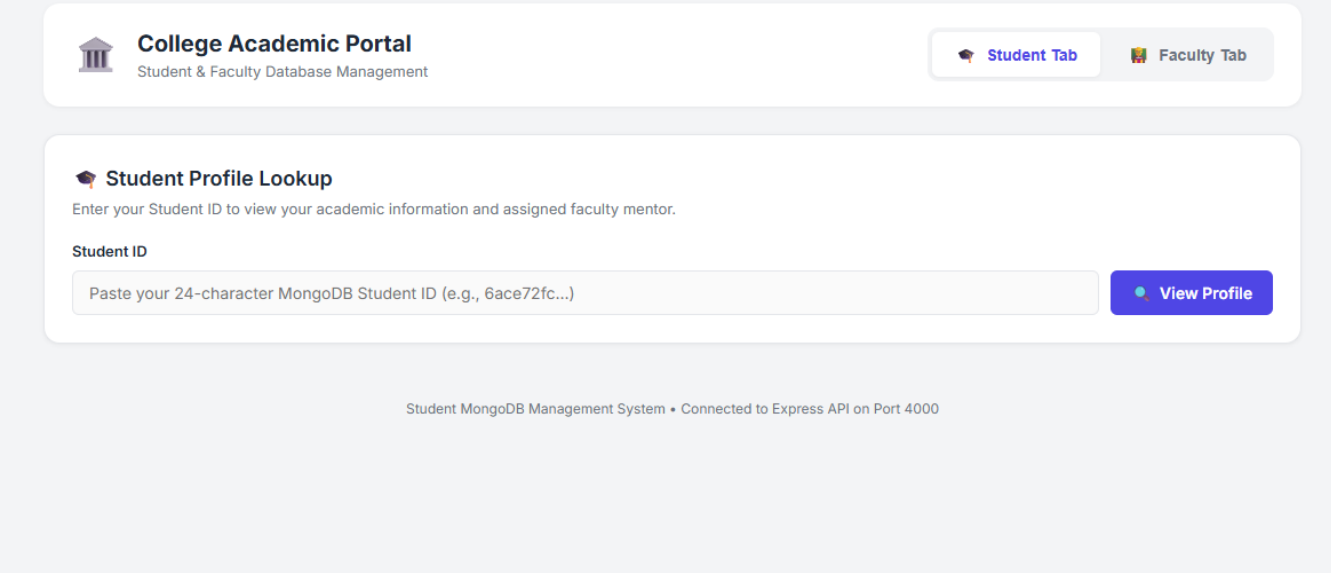


Screenshot 5: Faculty API

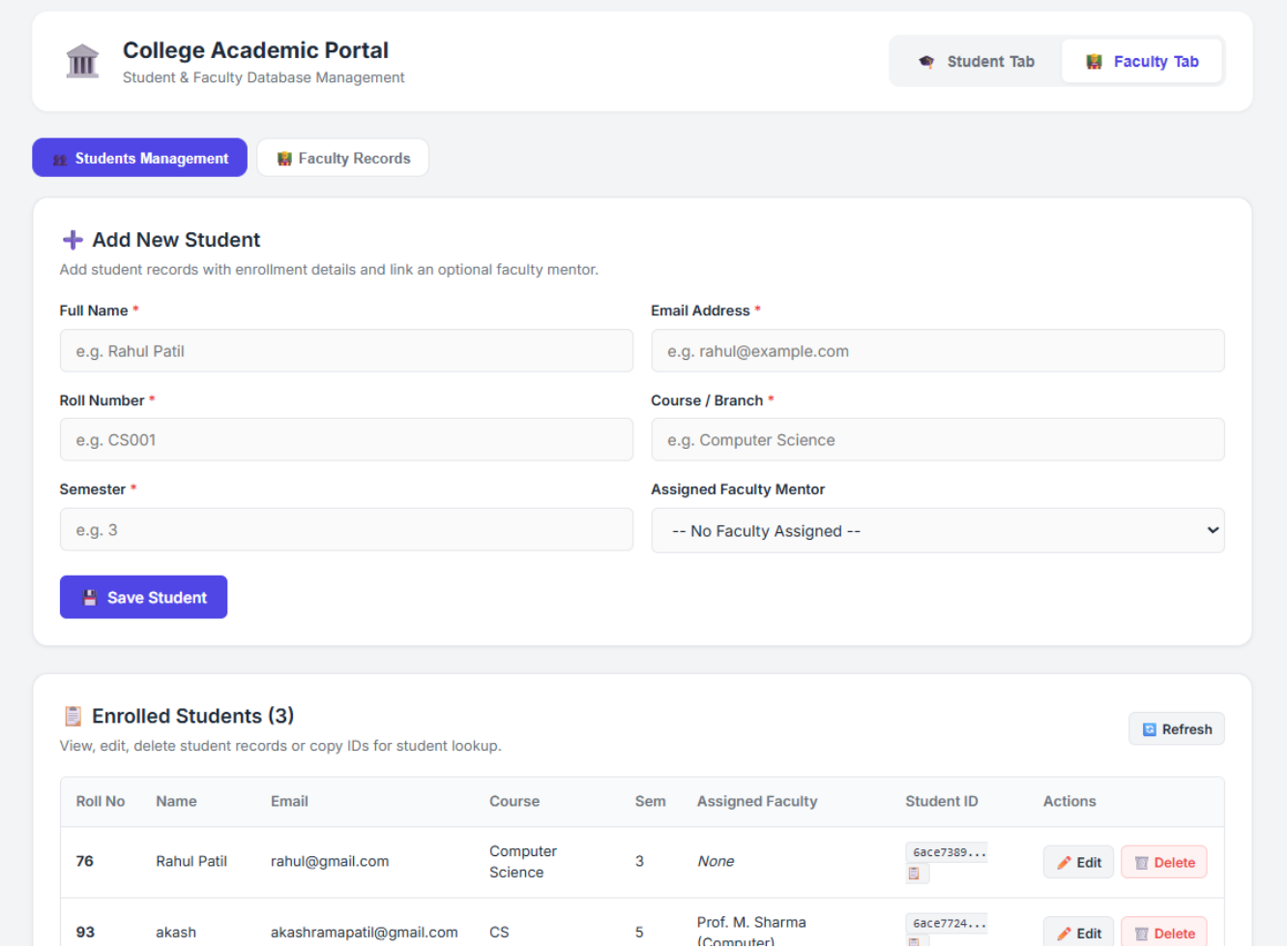


Screenshot 6: Frontend Student / Faculty tabs

Student Frontend:

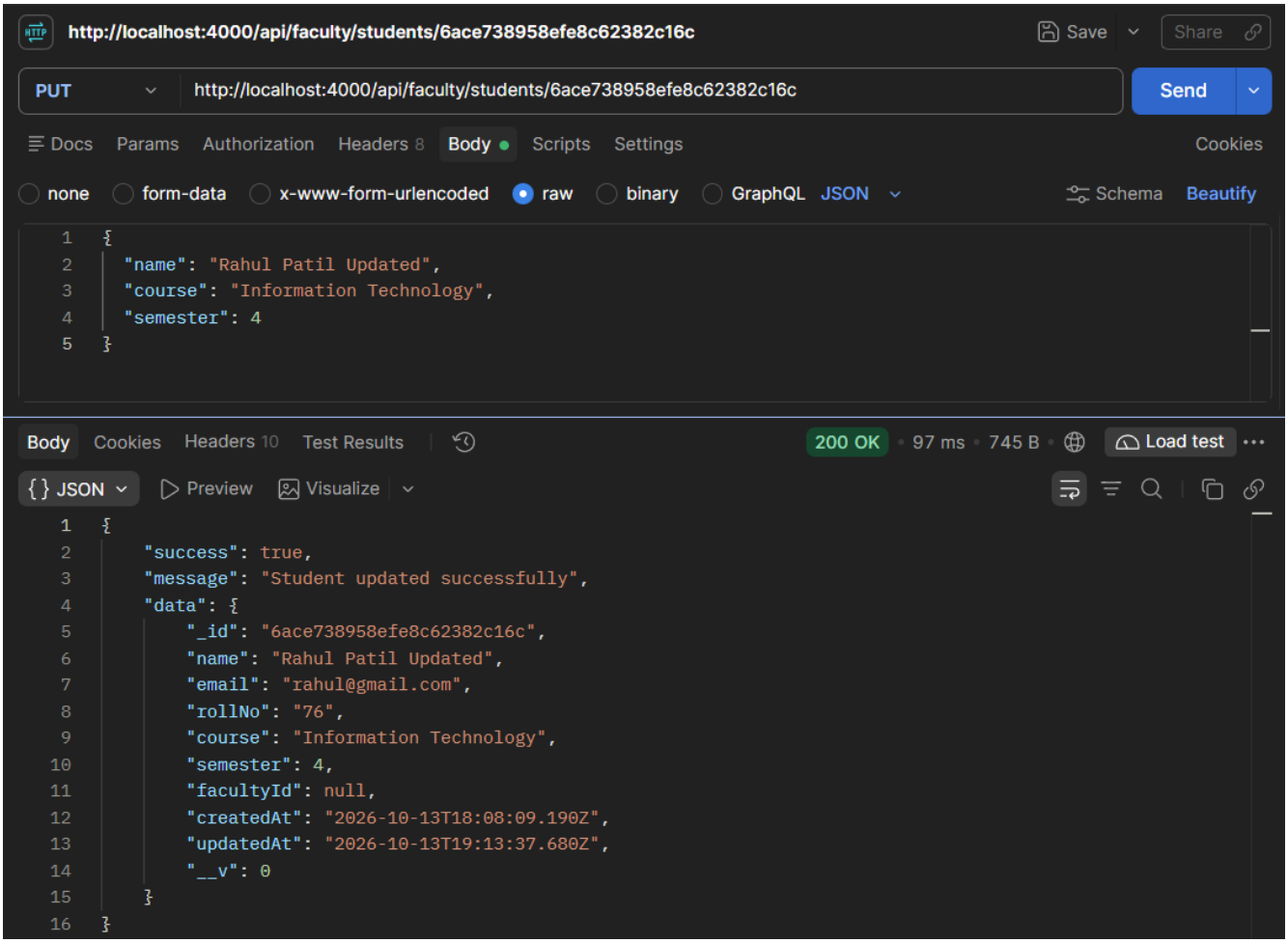


Faculty Frontend:

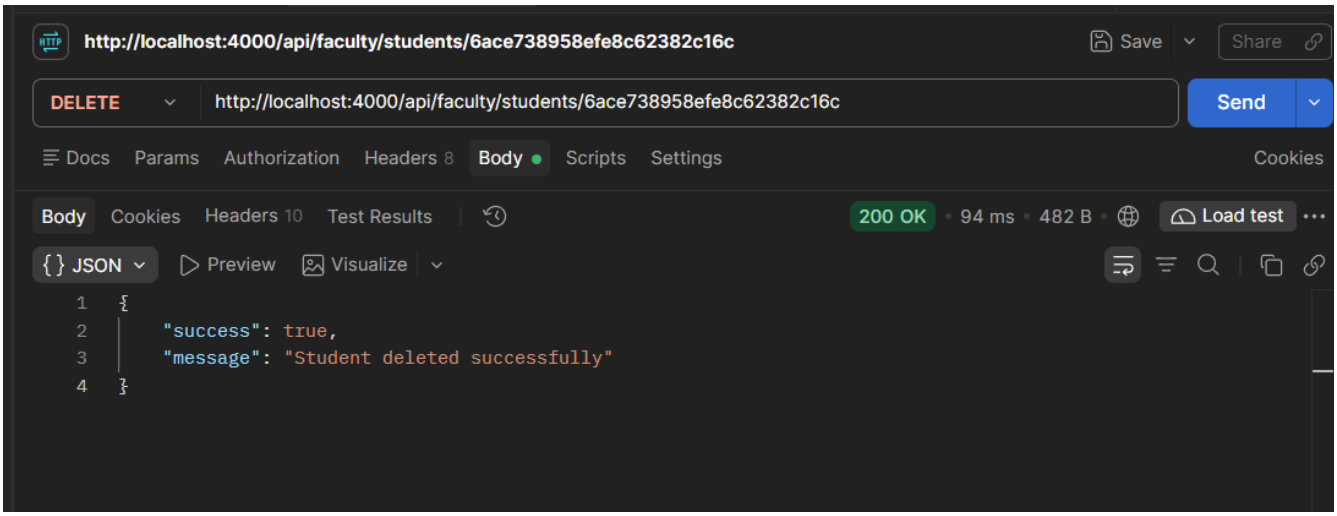


Screenshot 7: Update or delete operation

Update:



Delete:



13. Advantages

- Separates API routes, database models, and server configuration for easier maintenance.
- Uses a cloud database that can be accessed by a properly configured deployed application.
- REST endpoints make it possible to connect a browser-based frontend or another client.
- Mongoose supports schema validation and unique fields for data consistency.

14. Limitations

- Authentication and role-based authorization must be added before real student data is used.
- The application requires network access to connect to MongoDB Atlas.
- The exact available endpoints depend on the current route implementation.
- Deployment requires environment variables and suitable serverless configuration.

15. Future Enhancements

- Add secure login for administrators, faculty, and students.
- Add search, filtering, pagination, and sorting.
- Include attendance, marks, and course management.
- Add export to PDF or Excel.
- Deploy the frontend and API, configure production environment variables, and test the deployed URL.

16. Conclusion

This project demonstrates the core concepts of building a REST API with Node.js and Express.js and storing records in MongoDB through Mongoose. The Student and Faculty modules provide a base for managing academic records. After verifying the current endpoints, running the tests, and inserting actual screenshots, the report can be submitted as a practical assignment.

17. References

- Node.js documentation: https://nodejs.org/docs/
- Express.js documentation: https://expressjs.com/
- MongoDB documentation: https://www.mongodb.com/docs/
- Mongoose documentation: https://mongoosejs.com/docs/
- Postman learning center: https://learning.postman.com